

## LECTURE 22

# Memory Management Fundamentals

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

# In This Lecture

- Why physical memory sharing needs active management, and the hardware (base and limit registers) that enforces protection between processes
- Address binding — compile-time, load-time, and execution-time — and what each one fixes and when
- Logical vs. physical addresses, the MMU, and the relocation register — worked through with a numeric example
- Dynamic loading (load a routine only when called) and dynamic linking (defer linking a library until execution time), contrasted with static linking
- A brief look at how Windows resolves and loads DLLs at run time

# Why Memory Needs Managing

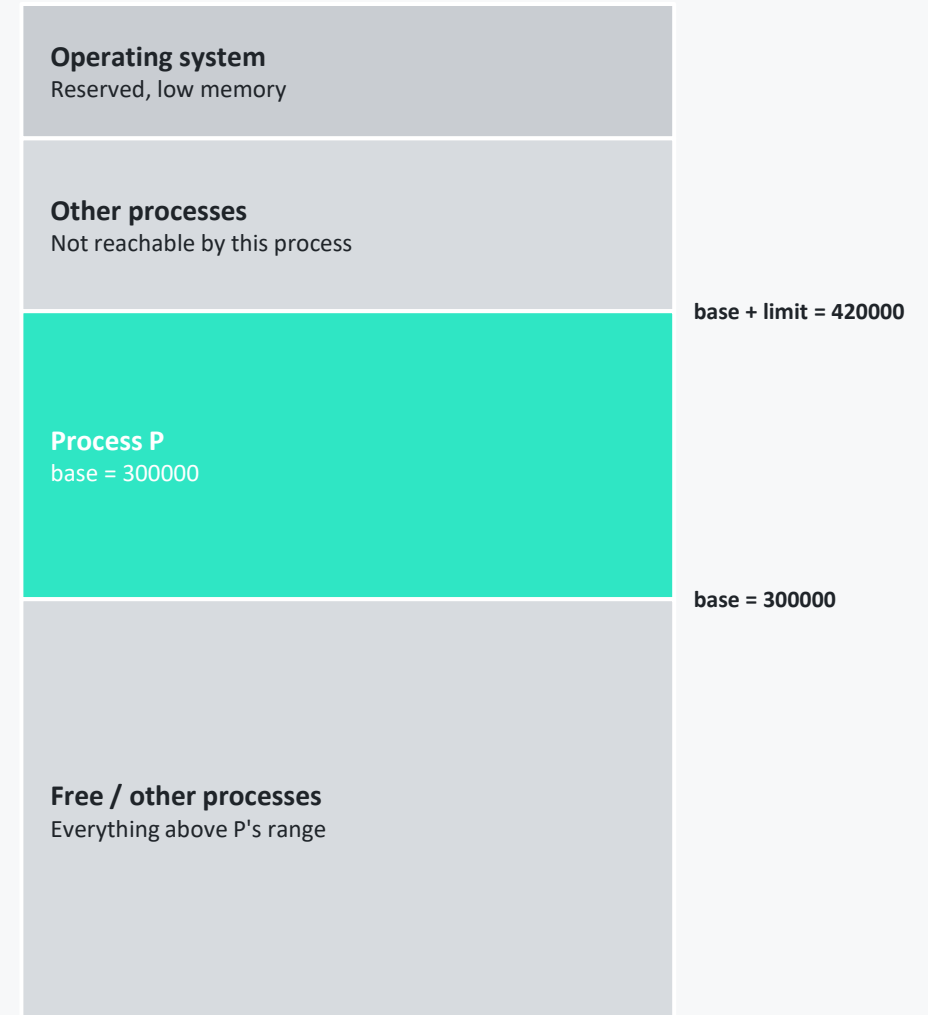
- Physical memory is a single, finite, shared resource — a machine with 16 GB of RAM has exactly 16 GB, whether running one program or fifty
- The operating system itself occupies some of it permanently (usually low memory, or the very top)
- **No process should read or write memory that belongs to another process, or to the OS** — whether by a bug or by design
- Delivering efficient sharing and airtight isolation at once is memory management's whole job — and it needs hardware help to do it fast enough

# Base and Limit Registers

- A base register holds the smallest legal physical address a process may touch; a limit register holds the size of its address range
- Memory protection is enforced by hardware: every address the CPU generates is compared against base/limit before reaching memory

**Valid range for P:  $300000 \leq \text{address} < 420000$  (limit = 120000)**

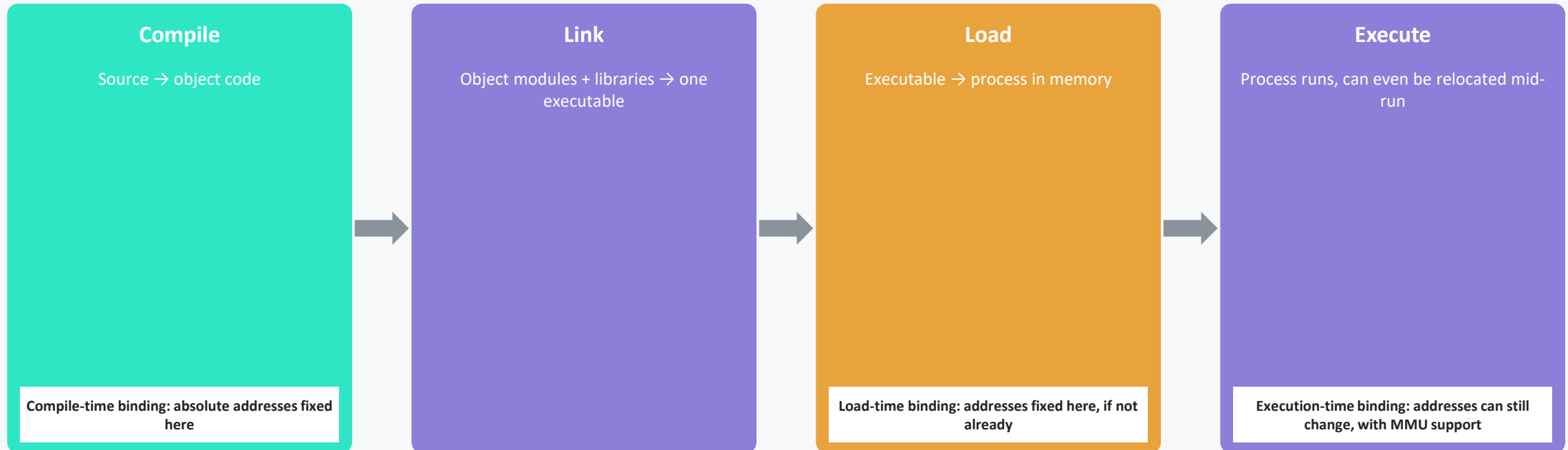
**Trap on violation** — An address outside  $[\text{base}, \text{base} + \text{limit})$  — e.g. 299999 or 420500 — is caught immediately by the hardware comparator, which traps to the OS. The OS then typically terminates the process with an addressing error — the origin of the segmentation-fault class of crashes. Only the OS, in kernel mode, may load new base/limit values.



# Address Binding

- **Compile-time binding** — absolute addresses fixed if the load location is known in advance. Most rigid: moving requires recompiling.
- **Load-time binding** — relocatable addresses computed once, when the program is loaded. Avoids recompiling, but cannot move again without reloading.
- **Execution-time (run-time) binding** — binding deferred until the program runs; addresses can change mid-run. Most flexible, but requires MMU hardware support.

# Address Binding Across a Program's Lifecycle



# Logical vs. Physical Addresses, and the MMU

- **Logical address (virtual address)** — the address the CPU generates while a program runs — what the program itself believes it's using.
- **Physical address** — the address that actually reaches the memory unit — the real location in physical RAM.
- Under compile-time and load-time binding these are identical. Under execution-time binding they deliberately differ, and translating one into the other on every access is the job of the Memory-Management Unit (MMU).

**The relocation register** — The simplest MMU design:  $\text{physical address} = \text{logical address} + \text{relocation register's value}$ . One register, one addition, applied on every single memory reference.

# Relocation Register: A Worked Example

- A process's relocation register holds 14000 — the OS decided to physically load this process starting at address 14000
- The program itself, compiled as if it would start at address 0, generates the logical address 346 to read one of its own local variables

$$\text{physical address} = 346 + 14000 = 14346$$

*The CPU never needed to know 14346 at all — it generated 346 exactly as its compiled code dictates, and the MMU transparently added the relocation offset. Only the OS, which sets the relocation register, knows where the process actually lives.*

# Dynamic Loading

- Not every routine needs to be in memory from the moment a program starts
- **Dynamic loading** keeps a routine on disk, in relocatable form, until it is actually called for the first time — only then does a small stub load it and transfer control to it
- Textbook example: error-handling code may account for a meaningful share of a program's total code size, yet run only on the rare occasion that error actually occurs
- Loading it unconditionally at startup wastes memory for the overwhelmingly common case where it's never needed — with dynamic loading, its memory cost is paid only if the error path is taken

# Dynamic Linking and Shared Libraries

- Static linking copies the complete machine code of every library routine directly into a program's executable at link time
- Dynamic linking postpones linking a library until execution time — the executable holds only a small stub that resolves and loads the real routine the first time it's called

## Static linking

Library code copied into every executable at link time — simple, but duplicated on disk and in RAM across every program that uses it

## Dynamic linking

A small stub resolves and loads the real library routine at execution time — one in-memory copy can be shared by every running program that needs it

Payoff: the OS keeps exactly one physical copy of a shared library in memory, and every process maps that same copy into its own address space. A library can also be upgraded without recompiling every dependent program.

# Windows Example: DLL Loading

- A .dll file holds compiled, shared code that one or more running programs can call into
- When a Windows program references a DLL function, the loader resolves it at run time: locates the DLL, maps its code into the process's address space if not already mapped, and resolves the function address
- If several programs depend on the same DLL at once, Windows keeps only one physical copy of its code resident in memory — every process maps its own view onto that same shared copy

**A DLL's code is shared; its data usually is not** — The instructions are identical for every caller, so one copy suffices — but each process gets its own private copy of any writable data the DLL uses.

# Key Takeaways

- Base and limit registers define each process's valid address range; hardware traps to the OS on any access outside it — protection without relying on software to behave
- Address binding happens at compile time (fixed, requires recompiling to move), load time (fixed once loaded), or execution time (can change while running, needs MMU)
- The logical address the CPU generates and the physical address the memory unit sees can differ; the MMU translates between them — the simplest MMU is a single relocation register
- Dynamic loading defers loading a routine until it's actually called, saving memory for rarely used code paths
- Dynamic linking defers linking a library until execution time via a stub, enabling shared libraries — one in-memory copy shared by every process that needs it
- Windows DLLs are a real-world instance of dynamic linking: one physical copy mapped into, and shared by, every running program that depends on it